

Student Management System

Generated by Doxygen 1.13.2

1 Hierarchical Index	1
1.1 Class Hierarchy	1
2 Class Index	3
2.1 Class List	3
3 File Index	5
3.1 File List	5
4 Class Documentation	7
4.1 Laikas Class Reference	7
4.1.1 Detailed Description	7
4.1.2 Constructor & Destructor Documentation	7
4.1.2.1 Laikas()	7
4.1.3 Member Function Documentation	8
4.1.3.1 baigti()	8
4.1.3.2 gautiLaikoSkirtuma()	8
4.1.3.3 pradeti()	8
4.1.4 Member Data Documentation	8
4.1.4.1 end	8
4.1.4.2 start	8
4.1.4.3 veiksmoPavadinimas	9
4.2 Studentas Class Reference	9
4.2.1 Detailed Description	11
4.2.2 Constructor & Destructor Documentation	11
4.2.2.1 Studentas() [1/5]	11
4.2.2.2 Studentas() [2/5]	11
4.2.2.3 Studentas() [3/5]	12
4.2.2.4 Studentas() [4/5]	12
4.2.2.5 Studentas() [5/5]	12
4.2.2.6 ~Studentas()	12
4.2.3 Member Function Documentation	12
4.2.3.1 addND()	12
4.2.3.2 clearND()	13
4.2.3.3 egzaminas()	13
4.2.3.4 galBalas()	13
4.2.3.5 galBalasMed()	13
4.2.3.6 galutinis()	14
4.2.3.7 nd()	14
4.2.3.8 nuskaitymasFile()	14
4.2.3.9 operator=() [1/2]	14
4.2.3.10 operator=() [2/2]	14
4.2.3.11 print()	15

4.2.3.12 read()	15
4.2.3.13 readStudent()	15
4.2.3.14 setEgzaminas()	15
4.2.3.15 setGalutinis()	16
4.2.3.16 skaiciuotiMed()	16
4.2.3.17 skaiciuotiVid()	16
4.2.4 Member Data Documentation	17
4.2.4.1 destruktoriuSk	17
4.2.4.2 egzaminas_	17
4.2.4.3 galutinis_	17
4.2.4.4 nd_	17
4.3 StudentasTest Class Reference	17
4.3.1 Member Function Documentation	18
4.3.1.1 SetUp()	18
4.3.2 Member Data Documentation	18
4.3.2.1 testStudent	18
4.4 Vector< T > Class Template Reference	18
4.4.1 Detailed Description	21
4.4.2 Member Typedef Documentation	21
4.4.2.1 const_iterator	21
4.4.2.2 const_pointer	21
4.4.2.3 const_reference	21
4.4.2.4 const_reverse_iterator	22
4.4.2.5 difference_type	22
4.4.2.6 iterator	22
4.4.2.7 pointer	22
4.4.2.8 reference	22
4.4.2.9 reverse_iterator	22
4.4.2.10 size_type	22
4.4.2.11 value_type	23
4.4.3 Constructor & Destructor Documentation	23
4.4.3.1 Vector() [1/5]	23
4.4.3.2 Vector() [2/5]	23
4.4.3.3 Vector() [3/5]	23
4.4.3.4 Vector() [4/5]	23
4.4.3.5 Vector() [5/5]	24
4.4.3.6 ~Vector()	24
4.4.4 Member Function Documentation	24
4.4.4.1 assign() [1/3]	24
4.4.4.2 assign() [2/3]	24
4.4.4.3 assign() [3/3]	25
4.4.4.4 at() [1/2]	25

4.4.4.5 at() [2/2]	25
4.4.4.6 back() [1/2]	25
4.4.4.7 back() [2/2]	26
4.4.4.8 begin() [1/2]	26
4.4.4.9 begin() [2/2]	26
4.4.4.10 capacity()	26
4.4.4.11 cbegin()	26
4.4.4.12 cend()	26
4.4.4.13 clear()	26
4.4.4.14 crbegin()	26
4.4.4.15 crend()	27
4.4.4.16 data() [1/2]	27
4.4.4.17 data() [2/2]	27
4.4.4.18 destroy_elements()	27
4.4.4.19 emplace_back()	27
4.4.4.20 empty()	28
4.4.4.21 end() [1/2]	28
4.4.4.22 end() [2/2]	28
4.4.4.23 erase() [1/2]	28
4.4.4.24 erase() [2/2]	28
4.4.4.25 front() [1/2]	28
4.4.4.26 front() [2/2]	28
4.4.4.27 insert() [1/3]	29
4.4.4.28 insert() [2/3]	29
4.4.4.29 insert() [3/3]	29
4.4.4.30 operator=() [1/3]	29
4.4.4.31 operator=() [2/3]	29
4.4.4.32 operator=() [3/3]	29
4.4.4.33 operator[]() [1/2]	30
4.4.4.34 operator[]() [2/2]	30
4.4.4.35 pop_back()	30
4.4.4.36 push_back() [1/2]	30
4.4.4.37 push_back() [2/2]	30
4.4.4.38 rbegin() [1/2]	30
4.4.4.39 rbegin() [2/2]	30
4.4.4.40 reallocate()	30
4.4.4.41 reallocations()	31
4.4.4.42 rend() [1/2]	31
4.4.4.43 rend() [2/2]	31
4.4.4.44 reserve()	31
4.4.4.45 resize() [1/2]	31
4.4.4.46 resize() [2/2]	31

4.4.4.47 shrink_to_fit()	32
4.4.4.48 size()	32
4.4.4.49 swap()	32
4.4.5 Friends And Related Symbol Documentation	32
4.4.5.1 operator!=	32
4.4.5.2 operator==	32
4.4.6 Member Data Documentation	32
4.4.6.1 capacity_	32
4.4.6.2 data_	33
4.4.6.3 realloc_count_	33
4.4.6.4 size_	33
4.5 VectorTest Class Reference	33
4.5.1 Member Function Documentation	33
4.5.1.1 expectEqual()	33
4.5.1.2 SetUp()	34
4.5.1.3 TearDown()	34
4.6 Zmogus Class Reference	34
4.6.1 Detailed Description	35
4.6.2 Constructor & Destructor Documentation	35
4.6.2.1 Zmogus() [1/4]	35
4.6.2.2 Zmogus() [2/4]	35
4.6.2.3 Zmogus() [3/4]	35
4.6.2.4 Zmogus() [4/4]	35
4.6.2.5 ~Zmogus()	36
4.6.3 Member Function Documentation	36
4.6.3.1 operator=() [1/2]	36
4.6.3.2 operator=() [2/2]	36
4.6.3.3 pavarde()	36
4.6.3.4 print()	36
4.6.3.5 read()	37
4.6.3.6 setPavarde()	37
4.6.3.7 setVardas()	37
4.6.3.8 vardas()	38
4.6.4 Member Data Documentation	38
4.6.4.1 pavarde_	38
4.6.4.2 vardas_	38
5 File Documentation	39
5.1 funkcijos.cpp File Reference	39
5.1.1 Function Documentation	40
5.1.1.1 compareByGalutinis()	40
5.1.1.2 compareByPavarde()	40

5.1.1.3 compareByVardas()	40
5.1.1.4 testuotiDuomenuApdorojima()	40
5.1.1.5 testuotiDuomenuApdorojimaImpl()	40
5.1.1.6 testuotiDuomenuApdorojimaImpl< std::vector< Studentas > >()	41
5.1.1.7 testuotiDuomenuApdorojimaImpl< Vector< Studentas > >()	41
5.1.1.8 testuotiStudentoMetodus()	41
5.1.1.9 testuotiZmogausKlase()	41
5.2 funkcijos.h File Reference	41
5.2.1 Function Documentation	42
5.2.1.1 compareByGalutinis()	42
5.2.1.2 compareByPavarde()	43
5.2.1.3 compareByVardas()	43
5.2.1.4 generuotiStudentus()	43
5.2.1.5 isvestiStudentusIFaila()	43
5.2.1.6 skaitytiIsFailo()	43
5.2.1.7 skirstytiStudentus()	44
5.2.1.8 sortStudentai()	44
5.2.1.9 testuotiDuomenuApdorojima()	44
5.2.1.10 testuotiDuomenuApdorojimaImpl()	45
5.2.1.11 testuotiStudentoMetodus()	45
5.2.1.12 testuotiZmogausKlase()	45
5.3 funkcijos.h	45
5.4 laikas.cpp File Reference	48
5.5 laikas.h File Reference	48
5.6 laikas.h	48
5.7 main.cpp File Reference	49
5.7.1 Function Documentation	49
5.7.1.1 main()	49
5.8 testai.cpp File Reference	49
5.8.1 Function Documentation	50
5.8.1.1 main()	50
5.8.1.2 TEST_F() [1/8]	50
5.8.1.3 TEST_F() [2/8]	50
5.8.1.4 TEST_F() [3/8]	50
5.8.1.5 TEST_F() [4/8]	50
5.8.1.6 TEST_F() [5/8]	51
5.8.1.7 TEST_F() [6/8]	51
5.8.1.8 TEST_F() [7/8]	51
5.8.1.9 TEST_F() [8/8]	51
5.9 vector.h File Reference	51
5.10 vector.h	52
5.11 vector_testai.cpp File Reference	53

5.11.1 Function Documentation	54
5.11.1.1 main()	54
5.11.1.2 TEST_F() [1/25]	54
5.11.1.3 TEST_F() [2/25]	54
5.11.1.4 TEST_F() [3/25]	54
5.11.1.5 TEST_F() [4/25]	55
5.11.1.6 TEST_F() [5/25]	55
5.11.1.7 TEST_F() [6/25]	55
5.11.1.8 TEST_F() [7/25]	55
5.11.1.9 TEST_F() [8/25]	55
5.11.1.10 TEST_F() [9/25]	55
5.11.1.11 TEST_F() [10/25]	55
5.11.1.12 TEST_F() [11/25]	55
5.11.1.13 TEST_F() [12/25]	56
5.11.1.14 TEST_F() [13/25]	56
5.11.1.15 TEST_F() [14/25]	56
5.11.1.16 TEST_F() [15/25]	56
5.11.1.17 TEST_F() [16/25]	56
5.11.1.18 TEST_F() [17/25]	56
5.11.1.19 TEST_F() [18/25]	56
5.11.1.20 TEST_F() [19/25]	56
5.11.1.21 TEST_F() [20/25]	57
5.11.1.22 TEST_F() [21/25]	57
5.11.1.23 TEST_F() [22/25]	57
5.11.1.24 TEST_F() [23/25]	57
5.11.1.25 TEST_F() [24/25]	57
5.11.1.26 TEST_F() [25/25]	57
5.12 zmogus.cpp File Reference	57
5.12.1 Function Documentation	58
5.12.1.1 operator<<()	58
5.12.1.2 operator>>()	58
5.13 zmogus.h File Reference	58
5.13.1 Function Documentation	59
5.13.1.1 operator<<()	59
5.13.1.2 operator>>()	59
5.14 zmogus.h	60
Index	61

Chapter 1

Hierarchical Index

1.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

Laikas	7
testing::Test	
StudentasTest	17
VectorTest	33
Vector< T >	18
Zmogus	34
Studentas	9

Chapter 2

Class Index

2.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

Laikas	Class for measuring execution time	7
Studentas	Student class that inherits from Zmogus (Person) abstract class	9
StudentasTest		17
Vector< T >	Dinaminis šabloninis konteineris, panašus į std::vector	18
VectorTest		33
Zmogus	Abstract base class representing a person	34

Chapter 3

File Index

3.1 File List

Here is a list of all files with brief descriptions:

funkcijos.cpp	39
funkcijos.h	41
laikas.cpp	48
laikas.h	48
main.cpp	49
testai.cpp	49
vector.h	51
vector_testai.cpp	53
zmogus.cpp	57
zmogus.h	58

Chapter 4

Class Documentation

4.1 Laikas Class Reference

Class for measuring execution time.

```
#include <laikas.h>
```

Public Member Functions

- [Laikas](#) (const std::string &pavadinimas)
Constructor.
- void [pradeti](#) ()
Start the timer Records the starting time point.
- void [baigti](#) ()
Stop the timer and print the elapsed time Records the ending time point and outputs the time difference.
- double [gautiLaikoSkirtuma](#) ()
Get the elapsed time in seconds.

Private Attributes

- std::chrono::high_resolution_clock::time_point [start](#)
Start time point.
- std::chrono::high_resolution_clock::time_point [end](#)
End time point.
- std::string [veiksmaPavadinimas](#)
Name of the operation being timed.

4.1.1 Detailed Description

Class for measuring execution time.

This class provides functionality to measure and report the execution time of code segments using high-resolution clock.

4.1.2 Constructor & Destructor Documentation

4.1.2.1 Laikas()

```
Laikas::Laikas (  
    const std::string & pavadinimas)
```

Constructor.

Parameters

<i>pavadinimas</i>	Name of the operation to time
--------------------	-------------------------------

4.1.3 Member Function Documentation

4.1.3.1 baigti()

```
void Laikas::baigti ()
```

Stop the timer and print the elapsed time Records the ending time point and outputs the time difference.

4.1.3.2 gautiLaikoSkirtuma()

```
double Laikas::gaustiLaikoSkirtuma ()
```

Get the elapsed time in seconds.

Returns

Time difference between start and end in seconds

4.1.3.3 pradeti()

```
void Laikas::pradeti ()
```

Start the timer Records the starting time point.

4.1.4 Member Data Documentation

4.1.4.1 end

```
std::chrono::high_resolution_clock::time_point Laikas::end [private]
```

End time point.

4.1.4.2 start

```
std::chrono::high_resolution_clock::time_point Laikas::start [private]
```

Start time point.

4.1.4.3 veiksmoPavadinimas

```
std::string Laikas::veiksmoPavadinimas [private]
```

Name of the operation being timed.

The documentation for this class was generated from the following files:

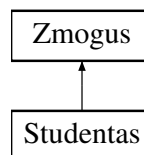
- [laikas.h](#)
- [laikas.cpp](#)

4.2 Studentas Class Reference

Student class that inherits from [Zmogus](#) (Person) abstract class.

```
#include <funkcijos.h>
```

Inheritance diagram for Studentas:



Public Member Functions

- [Studentas](#) ()
Default constructor Initializes a student with default values.
- [Studentas](#) (std::istream &is)
Constructor that initializes a student from input stream.
- [Studentas](#) (const std::string &vardas, const std::string &pavarde)
Constructor with name and surname.
- [Studentas](#) (const [Studentas](#) &other)
Copy constructor.
- [Studentas](#) & operator= (const [Studentas](#) &other)
Copy assignment operator.
- [Studentas](#) ([Studentas](#) &&other) noexcept
Move constructor.
- [Studentas](#) & operator= ([Studentas](#) &&other) noexcept
Move assignment operator.
- [~Studentas](#) () override
Virtual destructor Overrides the pure virtual destructor from [Zmogus](#).
- void [print](#) (std::ostream &os) const override
Prints student information to output stream.
- void [read](#) (std::istream &is) override
Reads student information from input stream.
- std::vector< int > [nd](#) () const
Get homework grades.

- int `egzaminas` () const
Get exam grade.
- double `galutinis` () const
Get final grade.
- void `setEgzaminas` (int `egzaminas`)
Set exam grade.
- void `setGalutinis` (double `galutinis`)
Set final grade.
- std::istream & `readStudent` (std::istream &is)
Read student data from input stream.
- void `addND` (int pazymys)
Add a homework grade.
- void `clearND` ()
Clear all homework grades.
- double `galBalas` () const
Calculate final grade (average method)
- double `galBalasMed` () const
Calculate final grade (median method)

Public Member Functions inherited from `Zmogus`

- `Zmogus` ()=default
Default constructor.
- `Zmogus` (const std::string &`vardas`, const std::string &`pavarde`)
Constructor with name and surname.
- `Zmogus` (const `Zmogus` &other)
Copy constructor.
- `Zmogus` & `operator=` (const `Zmogus` &other)
Copy assignment operator.
- `Zmogus` (`Zmogus` &&other) noexcept
Move constructor.
- `Zmogus` & `operator=` (`Zmogus` &&other) noexcept
Move assignment operator.
- virtual `~Zmogus` ()=0
Virtual destructor Pure virtual destructor makes this class abstract.
- std::string `vardas` () const
Get first name.
- std::string `pavarde` () const
Get last name.
- void `setVardas` (const std::string &`vardas`)
Set first name.
- void `setPavarde` (const std::string &`pavarde`)
Set last name.

Static Public Member Functions

- `template<typename Container>`
`static double skaiciuotiVid (const Container &nd)`
Calculate average of homework grades.
- `template<typename Container>`
`static double skaiciuotiMed (Container nd)`
Calculate median of homework grades.
- `template<typename Container>`
`static void nuskaitymasFile (Container &grupe, const std::string &failoPavadinimas)`
Read students from file (static method)

Static Public Attributes

- `static int destruktoriuSk = 0`
Static counter for tracking destructor calls (for testing)

Private Attributes

- `std::vector< int > nd_`
Vector of homework (namų darbų) scores.
- `int egzaminas_`
Exam score.
- `double galutinis_`
Final grade.

Additional Inherited Members

Protected Attributes inherited from [Zmogus](#)

- `std::string vardas_`
First name of the person.
- `std::string pavarde_`
Last name of the person.

4.2.1 Detailed Description

Student class that inherits from [Zmogus](#) (Person) abstract class.

This class represents a student with homework grades, exam score and final grade. It inherits from the abstract [Zmogus](#) class and implements its pure virtual methods.

4.2.2 Constructor & Destructor Documentation

4.2.2.1 Studentas() [1/5]

```
Studentas::Studentas () [inline]
```

Default constructor Initializes a student with default values.

4.2.2.2 Studentas() [2/5]

```
Studentas::Studentas (
    std::istream & is)
```

Constructor that initializes a student from input stream.

Parameters

<i>is</i>	Input stream to read from
-----------	---------------------------

4.2.2.3 Studentas() [3/5]

```
Studentas::Studentas (  
    const std::string & vardas,  
    const std::string & pavarde) [inline]
```

Constructor with name and surname.

Parameters

<i>vardas</i>	Student's first name
<i>pavarde</i>	Student's last name

4.2.2.4 Studentas() [4/5]

```
Studentas::Studentas (  
    const Studentas & other)
```

Copy constructor.

Parameters

<i>other</i>	Another student to copy from
--------------	------------------------------

4.2.2.5 Studentas() [5/5]

```
Studentas::Studentas (  
    Studentas && other) [noexcept]
```

Move constructor.

Parameters

<i>other</i>	Another student to move from
--------------	------------------------------

4.2.2.6 ~Studentas()

```
Studentas::~~Studentas () [override]
```

Virtual destructor Overrides the pure virtual destructor from [Zmogus](#).

4.2.3 Member Function Documentation**4.2.3.1 addND()**

```
void Studentas::addND (  
    int pazymys)
```

Add a homework grade.

Parameters

<i>pazymys</i>	Grade to add
----------------	--------------

Exceptions

<i>std::invalid_argument</i>	If grade is not between 1 and 10
------------------------------	----------------------------------

4.2.3.2 clearND()

```
void Studentas::clearND ()
```

Clear all homework grades.

4.2.3.3 egzaminas()

```
int Studentas::egzaminas () const [inline]
```

Get exam grade.

Returns

Exam grade

4.2.3.4 galBalas()

```
double Studentas::galBalas () const [inline]
```

Calculate final grade (average method)

Returns

Final grade based on homework average and exam

4.2.3.5 galBalasMed()

```
double Studentas::galBalasMed () const [inline]
```

Calculate final grade (median method)

Returns

Final grade based on homework median and exam

4.2.3.6 galutinis()

```
double Studentas::galutinis () const [inline]
```

Get final grade.

Returns

Final grade

4.2.3.7 nd()

```
std::vector< int > Studentas::nd () const [inline]
```

Get homework grades.

Returns

[Vector](#) of homework grades

4.2.3.8 nuskaitymasFile()

```
template<typename Container>
static void Studentas::nuskaitymasFile (
    Container & grupe,
    const std::string & failoPavadinimas) [inline], [static]
```

Read students from file (static method)

Parameters

<i>grupe</i>	Container to store students
<i>failoPavadinimas</i>	Name of the file to read from

4.2.3.9 operator=() [1/2]

```
Studentas & Studentas::operator= (
    const Studentas & other)
```

Copy assignment operator.

Parameters

<i>other</i>	Another student to copy from
--------------	------------------------------

Returns

Reference to this student after assignment

4.2.3.10 operator=() [2/2]

```
Studentas & Studentas::operator= (
    Studentas && other) [noexcept]
```

Move assignment operator.

Parameters

<i>other</i>	Another student to move from
--------------	------------------------------

Returns

Reference to this student after assignment

4.2.3.11 print()

```
void Studentas::print (  
    std::ostream & os) const [override], [virtual]
```

Prints student information to output stream.

Parameters

<i>os</i>	Output stream to print to
-----------	---------------------------

Implements [Zmogus](#).

4.2.3.12 read()

```
void Studentas::read (  
    std::istream & is) [override], [virtual]
```

Reads student information from input stream.

Parameters

<i>is</i>	Input stream to read from
-----------	---------------------------

Implements [Zmogus](#).

4.2.3.13 readStudent()

```
std::istream & Studentas::readStudent (  
    std::istream & is)
```

Read student data from input stream.

Parameters

<i>is</i>	Input stream to read from
-----------	---------------------------

Returns

Reference to the input stream

4.2.3.14 setEgzaminas()

```
void Studentas::setEgzaminas (  
    int egzaminas) [inline]
```

Set exam grade.

Parameters

<i>egzaminas</i>	Exam grade to set
------------------	-------------------

4.2.3.15 setGalutinis()

```
void Studentas::setGalutinis (  
    double galutinis) [inline]
```

Set final grade.

Parameters

<i>galutinis</i>	Final grade to set
------------------	--------------------

4.2.3.16 skaiciuotiMed()

```
template<typename Container>  
static double Studentas::skaiciuotiMed (  
    Container nd) [inline], [static]
```

Calculate median of homework grades.

Parameters

<i>nd</i>	Container of grades (will be sorted)
-----------	--------------------------------------

Returns

Median value

4.2.3.17 skaiciuotiVid()

```
template<typename Container>  
static double Studentas::skaiciuotiVid (  
    const Container & nd) [inline], [static]
```

Calculate average of homework grades.

Parameters

<i>nd</i>	Container of grades
-----------	---------------------

Returns

Average value

4.2.4 Member Data Documentation

4.2.4.1 destruktoriuSk

```
int Studentas::destruktoriuSk = 0 [static]
```

Static counter for tracking destructor calls (for testing)

4.2.4.2 egzaminas_

```
int Studentas::egzaminas_ [private]
```

Exam score.

4.2.4.3 galutinis_

```
double Studentas::galutinis_ [private]
```

Final grade.

4.2.4.4 nd_

```
std::vector<int> Studentas::nd_ [private]
```

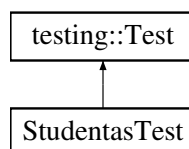
[Vector](#) of homework (namų darbų) scores.

The documentation for this class was generated from the following files:

- [funkcijos.h](#)
- [funkcijos.cpp](#)

4.3 StudentasTest Class Reference

Inheritance diagram for StudentasTest:



Protected Member Functions

- void [SetUp](#) () override

Protected Attributes

- [Studentas testStudent](#)

4.3.1 Member Function Documentation

4.3.1.1 SetUp()

```
void StudentasTest::SetUp () [inline], [override], [protected]
```

4.3.2 Member Data Documentation

4.3.2.1 testStudent

```
Studentas StudentasTest::testStudent [protected]
```

The documentation for this class was generated from the following file:

- [testai.cpp](#)

4.4 Vector< T > Class Template Reference

Dinaminis šabloninis konteineris, panašus į `std::vector`.

```
#include <vector.h>
```

Public Types

- using [value_type](#) = T
Elemento tipas.
- using [size_type](#) = `std::size_t`
Tipas dydžio žymėjimui.
- using [difference_type](#) = `std::ptrdiff_t`
Skirtumo tipas tarp iteratorių
- using [reference](#) = T&
Nuoroda į elementą
- using [const_reference](#) = `const T&`
Konstanta nuoroda į elementą
- using [pointer](#) = T*
Rodyklė į elementą
- using [const_pointer](#) = `const T*`
Konstanta rodyklė į elementą
- using [iterator](#) = T*
Iteratorius.
- using [const_iterator](#) = `const T*`
Konstanta iteratorius.
- using [reverse_iterator](#) = `std::reverse_iterator<iterator>`
Atvirkštinis iteratorius.
- using [const_reverse_iterator](#) = `std::reverse_iterator<const_iterator>`
Konstanta atvirkštinis iteratorius.

Public Member Functions

- `Vector ()`=default
Numatytasis konstruktorius.
- `Vector (size_type count, const T &value=T())`
Konstruktorius su elementų skaičiumi ir pradine verte.
- `Vector (std::initializer_list< T > init)`
Konstruktorius iš initializer_list.
- `Vector (const Vector &other)`
Kopijavimo konstruktorius.
- `Vector (Vector &&other) noexcept`
Perkėlimo konstruktorius.
- `~Vector ()`
Destruktorius.
- `Vector & operator= (const Vector &other)`
Kopijavimo operatorius.
- `Vector & operator= (Vector &&other) noexcept`
Perkėlimo operatorius.
- `Vector & operator= (std::initializer_list< T > ilist)`
Priskyrimas iš initializer_list.
- `reference at (size_type pos)`
Prieiga su ribų tikrinimu.
- `const_reference at (size_type pos) const`
Prieiga su ribų tikrinimu (konstanta).
- `reference operator[] (size_type pos)`
Prieiga be ribų tikrinimo.
- `const_reference operator[] (size_type pos) const`
Prieiga be ribų tikrinimo (konstanta).
- `reference front ()`
Pirmas elementas.
- `const_reference front () const`
Pirmas elementas (konstanta).
- `reference back ()`
Paskutinis elementas.
- `const_reference back () const`
Paskutinis elementas (konstanta).
- `pointer data () noexcept`
Rodyklė į duomenis.
- `const_pointer data () const noexcept`
Konstanta rodyklė į duomenis.
- `iterator begin () noexcept`
- `const_iterator begin () const noexcept`
- `const_iterator cbegin () const noexcept`
- `iterator end () noexcept`
- `const_iterator end () const noexcept`
- `const_iterator cend () const noexcept`
- `reverse_iterator rbegin () noexcept`
- `const_reverse_iterator rbegin () const noexcept`
- `const_reverse_iterator crbegin () const noexcept`
- `reverse_iterator rend () noexcept`
- `const_reverse_iterator rend () const noexcept`
- `const_reverse_iterator crend () const noexcept`

- `bool empty ()` `const noexcept`
Tikrina ar vektorius tuščias.
- `size_type size ()` `const noexcept`
Elementų skaičius.
- `size_type capacity ()` `const noexcept`
Esama talpa.
- `void reserve (size_type new_cap)`
Rezervuoja atmintį
- `void shrink_to_fit ()`
Sumažina talpą iki dydžio.
- `void clear ()` `noexcept`
Ištrina visus elementus.
- `iterator insert (const_iterator pos, const T &value)`
Įterpia vieną elementą
- `iterator insert (const_iterator pos, T &&value)`
Įterpia vieną elementą (perkėlimas)
- `iterator insert (const_iterator pos, size_type count, const T &value)`
Įterpia kelis.
- `iterator erase (const_iterator pos)`
Ištrina vieną
- `iterator erase (const_iterator first, const_iterator last)`
Ištrina intervalą
- `void push_back (const T &value)`
Prideda gale.
- `void push_back (T &&value)`
Prideda gale (perkėlimas)
- `template<class... Args>`
`reference emplace_back (Args &&... args)`
Sukuria objektą gale, panaudojant nurodytus argumentus.
- `void pop_back ()`
Pašalina paskutinį elementą
- `void resize (size_type count)`
Keičia dydį (numatyta reikšmė)
- `void resize (size_type count, const T &value)`
Keičia dydį su reikšme.
- `void swap (Vector &other)` `noexcept`
Pakeičia du vektorius vietomis.
- `void assign (size_type count, const T &value)`
Priskiria reikšmes.
- `template<class InputIt>`
`void assign (InputIt first, InputIt last)`
Priskiria iš intervalo.
- `void assign (std::initializer_list< T > ilist)`
Priskiria iš sąrašo.
- `size_t reallocations ()` `const`
Gauk kiek kartų buvo atlikta atminties persikirstymų.

Private Member Functions

- `void reallocate (size_type new_capacity)`
Atminties persikirstymas naujai talpai.
- `void destroy_elements ()`
Naikina visus elementus.

Private Attributes

- `pointer data_` = nullptr
Rodyklė į duomenų masyvą
- `size_type size_` = 0
Dabartinis elementų skaičius.
- `size_type capacity_` = 0
Dabartinė talpa (alokacijos dydis)
- `size_t realloc_count_` = 0
Kiek kartų buvo perskirstyta atmintis.

Friends

- `bool operator==` (const `Vector` &lhs, const `Vector` &rhs)
- `bool operator!=` (const `Vector` &lhs, const `Vector` &rhs)

4.4.1 Detailed Description

```
template<typename T>
class Vector< T >
```

Dinaminis šabloninis konteineris, panašus į `std::vector`.

Template Parameters

<code>T</code>	Elementų tipas.
----------------	-----------------

4.4.2 Member Typedef Documentation

4.4.2.1 const_iterator

```
template<typename T>
using Vector< T >::const_iterator = const T*
```

Konstanta iteratorius.

4.4.2.2 const_pointer

```
template<typename T>
using Vector< T >::const_pointer = const T*
```

Konstanta rodyklė į elementą

4.4.2.3 const_reference

```
template<typename T>
using Vector< T >::const_reference = const T&
```

Konstanta nuoroda į elementą

4.4.2.4 const_reverse_iterator

```
template<typename T>
using Vector< T >::const_reverse_iterator = std::reverse_iterator<const_iterator>
```

Konstanta atvirkštinis iteratorius.

4.4.2.5 difference_type

```
template<typename T>
using Vector< T >::difference_type = std::ptrdiff_t
```

Skirtumo tipas tarp iteratorių

4.4.2.6 iterator

```
template<typename T>
using Vector< T >::iterator = T*
```

Iteratorius.

4.4.2.7 pointer

```
template<typename T>
using Vector< T >::pointer = T*
```

Rodyklė į elementą

4.4.2.8 reference

```
template<typename T>
using Vector< T >::reference = T&
```

Nuoroda į elementą

4.4.2.9 reverse_iterator

```
template<typename T>
using Vector< T >::reverse_iterator = std::reverse_iterator<iterator>
```

Atvirkštinis iteratorius.

4.4.2.10 size_type

```
template<typename T>
using Vector< T >::size_type = std::size_t
```

Tipas dydžio žymėjimui.

4.4.2.11 value_type

```
template<typename T>
using Vector< T >::value_type = T
```

Elemento tipas.

4.4.3 Constructor & Destructor Documentation

4.4.3.1 Vector() [1/5]

```
template<typename T>
Vector< T >::Vector () [default]
```

Numatytasis konstruktorius.

4.4.3.2 Vector() [2/5]

```
template<typename T>
Vector< T >::Vector (
    size_type count,
    const T & value = T()) [explicit]
```

Konstruktorius su elementų skaičiumi ir pradine verte.

Parameters

<i>count</i>	Elementų kiekis.
<i>value</i>	Pradinė reikšmė kiekvienam elementui.

4.4.3.3 Vector() [3/5]

```
template<typename T>
Vector< T >::Vector (
    std::initializer_list< T > init)
```

Konstruktorius iš initializer_list.

Parameters

<i>init</i>	Inicilizacijos sąrašas.
-------------	-------------------------

4.4.3.4 Vector() [4/5]

```
template<typename T>
Vector< T >::Vector (
    const Vector< T > & other)
```

Kopijavimo konstruktorius.

Parameters

<i>other</i>	Kitas vektorius.
--------------	------------------

4.4.3.5 Vector() [5/5]

```
template<typename T>
Vector< T >::Vector (
    Vector< T > && other) [noexcept]
```

Perkėlimo konstruktorius.

Parameters

<i>other</i>	Perkeliamas vektorius.
--------------	------------------------

4.4.3.6 ~Vector()

```
template<typename T>
Vector< T >::~~Vector ()
```

Destruktorius.

4.4.4 Member Function Documentation**4.4.4.1 assign()** [1/3]

```
template<typename T>
template<class InputIt>
void Vector< T >::assign (
    InputIt first,
    InputIt last)
```

Priskiria iš intervalo.

4.4.4.2 assign() [2/3]

```
template<typename T>
void Vector< T >::assign (
    size_type count,
    const T & value)
```

Priskiria reikšmes.

4.4.4.3 assign() [3/3]

```
template<typename T>
void Vector< T >::assign (
    std::initializer_list< T > ilist)
```

Priskiria iš sąrašo.

4.4.4.4 at() [1/2]

```
template<typename T>
reference Vector< T >::at (
    size_type pos)
```

Prieiga su ribų tikrinimu.

Parameters

<i>pos</i>	Indeksas.
------------	-----------

Returns

Nuoroda į elementą.

4.4.4.5 at() [2/2]

```
template<typename T>
const_reference Vector< T >::at (
    size_type pos) const
```

Prieiga su ribų tikrinimu (konstanta).

Parameters

<i>pos</i>	Indeksas.
------------	-----------

Returns

Konstanta nuoroda į elementą.

4.4.4.6 back() [1/2]

```
template<typename T>
reference Vector< T >::back ()
```

Paskutinis elementas.

4.4.4.7 back() [2/2]

```
template<typename T>
const_reference Vector< T >::back () const
```

Paskutinis elementas (konstanta)

4.4.4.8 begin() [1/2]

```
template<typename T>
const_iterator Vector< T >::begin () const [noexcept]
```

4.4.4.9 begin() [2/2]

```
template<typename T>
iterator Vector< T >::begin () [noexcept]
```

4.4.4.10 capacity()

```
template<typename T>
size_type Vector< T >::capacity () const [noexcept]
```

Esama talpa.

4.4.4.11 cbegin()

```
template<typename T>
const_iterator Vector< T >::cbegin () const [noexcept]
```

4.4.4.12 cend()

```
template<typename T>
const_iterator Vector< T >::cend () const [noexcept]
```

4.4.4.13 clear()

```
template<typename T>
void Vector< T >::clear () [noexcept]
```

Ištrina visus elementus.

4.4.4.14 crbegin()

```
template<typename T>
const_reverse_iterator Vector< T >::crbegin () const [noexcept]
```

4.4.4.15 crend()

```
template<typename T>
const_reverse_iterator Vector< T >::crend () const [noexcept]
```

4.4.4.16 data() [1/2]

```
template<typename T>
const_pointer Vector< T >::data () const [noexcept]
```

Konstanta rodyklė į duomenis.

4.4.4.17 data() [2/2]

```
template<typename T>
pointer Vector< T >::data () [noexcept]
```

Rodyklė į duomenis.

4.4.4.18 destroy_elements()

```
template<typename T>
void Vector< T >::destroy_elements () [private]
```

Naikina visus elementus.

4.4.4.19 emplace_back()

```
template<typename T>
template<class... Args>
reference Vector< T >::emplace_back (
    Args &&... args)
```

Sukuria objektą gale, panaudojant nurodytus argumentus.

Template Parameters

<i>Args</i>	Argumentų tipai.
-------------	------------------

Parameters

<i>args</i>	Argumentai, naudojami sukurti objektui.
-------------	---

Returns

Nuoroda į naują objektą.

4.4.4.20 empty()

```
template<typename T>
bool Vector< T >::empty () const [noexcept]
```

Tikrina ar vektorius tuščias.

4.4.4.21 end() [1/2]

```
template<typename T>
const_iterator Vector< T >::end () const [noexcept]
```

4.4.4.22 end() [2/2]

```
template<typename T>
iterator Vector< T >::end () [noexcept]
```

4.4.4.23 erase() [1/2]

```
template<typename T>
iterator Vector< T >::erase (
    const_iterator first,
    const_iterator last)
```

Ištrina intervalą

4.4.4.24 erase() [2/2]

```
template<typename T>
iterator Vector< T >::erase (
    const_iterator pos)
```

Ištrina vieną

4.4.4.25 front() [1/2]

```
template<typename T>
reference Vector< T >::front ()
```

Pirmas elementas.

4.4.4.26 front() [2/2]

```
template<typename T>
const_reference Vector< T >::front () const
```

Pirmas elementas (konstanta)

4.4.4.27 insert() [1/3]

```
template<typename T>
iterator Vector< T >::insert (
    const_iterator pos,
    const T & value)
```

Įterpia vieną elementą

4.4.4.28 insert() [2/3]

```
template<typename T>
iterator Vector< T >::insert (
    const_iterator pos,
    size_type count,
    const T & value)
```

Įterpia kelis.

4.4.4.29 insert() [3/3]

```
template<typename T>
iterator Vector< T >::insert (
    const_iterator pos,
    T && value)
```

Įterpia vieną elementą (perkėlimas)

4.4.4.30 operator=() [1/3]

```
template<typename T>
Vector & Vector< T >::operator= (
    const Vector< T > & other)
```

Kopijavimo operatorius.

4.4.4.31 operator=() [2/3]

```
template<typename T>
Vector & Vector< T >::operator= (
    std::initializer_list< T > ilist)
```

Priskyrimas iš initializer_list.

4.4.4.32 operator=() [3/3]

```
template<typename T>
Vector & Vector< T >::operator= (
    Vector< T > && other) [noexcept]
```

Perkėlimo operatorius.

4.4.4.33 operator[]() [1/2]

```
template<typename T>
reference Vector< T >::operator[] (
    size_type pos)
```

Prieiga be ribų tikrinimo.

4.4.4.34 operator[]() [2/2]

```
template<typename T>
const_reference Vector< T >::operator[] (
    size_type pos) const
```

Prieiga be ribų tikrinimo (konstanta)

4.4.4.35 pop_back()

```
template<typename T>
void Vector< T >::pop_back ()
```

Pašalina paskutinį elementą

4.4.4.36 push_back() [1/2]

```
template<typename T>
void Vector< T >::push_back (
    const T & value)
```

Prideda gale.

4.4.4.37 push_back() [2/2]

```
template<typename T>
void Vector< T >::push_back (
    T && value)
```

Prideda gale (perkėlimas)

4.4.4.38 rbegin() [1/2]

```
template<typename T>
const_reverse_iterator Vector< T >::rbegin () const [noexcept]
```

4.4.4.39 rbegin() [2/2]

```
template<typename T>
reverse_iterator Vector< T >::rbegin () [noexcept]
```

4.4.4.40 reallocate()

```
template<typename T>
void Vector< T >::reallocate (
    size_type new_capacity) [private]
```

Atminties perskirstymas naujai talpai.

Parameters

<code>new_capacity</code>	Nauja talpa.
---------------------------	--------------

4.4.4.41 reallocations()

```
template<typename T>
size_t Vector< T >::reallocations () const
```

Gauk kiek kartų buvo atlikta atminties perskirstymų.

Returns

Perskirstymų skaičius.

4.4.4.42 rend() [1/2]

```
template<typename T>
const_reverse_iterator Vector< T >::rend () const [noexcept]
```

4.4.4.43 rend() [2/2]

```
template<typename T>
reverse_iterator Vector< T >::rend () [noexcept]
```

4.4.4.44 reserve()

```
template<typename T>
void Vector< T >::reserve (
    size_type new_cap)
```

Rezervuoja atmintį

4.4.4.45 resize() [1/2]

```
template<typename T>
void Vector< T >::resize (
    size_type count)
```

Keičia dydį (numatyta reikšmė)

4.4.4.46 resize() [2/2]

```
template<typename T>
void Vector< T >::resize (
    size_type count,
    const T & value)
```

Keičia dydį su reikšme.

4.4.4.47 shrink_to_fit()

```
template<typename T>
void Vector< T >::shrink_to_fit ()
```

Sumažina talpą iki dydžio.

4.4.4.48 size()

```
template<typename T>
size_type Vector< T >::size () const [noexcept]
```

Elementų skaičius.

4.4.4.49 swap()

```
template<typename T>
void Vector< T >::swap (
    Vector< T > & other) [noexcept]
```

Pakeičia du vektorius vietomis.

4.4.5 Friends And Related Symbol Documentation

4.4.5.1 operator"!="

```
template<typename T>
bool operator!= (
    const Vector< T > & lhs,
    const Vector< T > & rhs) [friend]
```

4.4.5.2 operator==

```
template<typename T>
bool operator== (
    const Vector< T > & lhs,
    const Vector< T > & rhs) [friend]
```

4.4.6 Member Data Documentation

4.4.6.1 capacity_

```
template<typename T>
size_type Vector< T >::capacity_ = 0 [private]
```

Dabartinė talpa (alokacijos dydis)

4.4.6.2 data_

```
template<typename T>
pointer Vector< T >::data_ = nullptr [private]
```

Rodyklė į duomenų masyvą

4.4.6.3 realloc_count_

```
template<typename T>
size_t Vector< T >::realloc_count_ = 0 [private]
```

Kiek kartų buvo perskirstyta atmintis.

4.4.6.4 size_

```
template<typename T>
size_type Vector< T >::size_ = 0 [private]
```

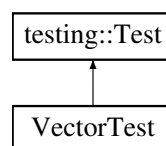
Dabartinis elementų skaičius.

The documentation for this class was generated from the following file:

- [vector.h](#)

4.5 VectorTest Class Reference

Inheritance diagram for VectorTest:



Protected Member Functions

- void [SetUp](#) () override
- void [TearDown](#) () override
- template<typename T>
void [expectEqual](#) (const [Vector](#)< T > &v1, const std::vector< T > &v2)

4.5.1 Member Function Documentation

4.5.1.1 expectEqual()

```
template<typename T>
void VectorTest::expectEqual (
    const Vector< T > & v1,
    const std::vector< T > & v2) [inline], [protected]
```

4.5.1.2 SetUp()

```
void VectorTest::SetUp () [inline], [override], [protected]
```

4.5.1.3 TearDown()

```
void VectorTest::TearDown () [inline], [override], [protected]
```

The documentation for this class was generated from the following file:

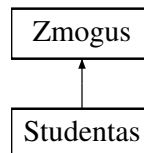
- [vector_testai.cpp](#)

4.6 Zmogus Class Reference

Abstract base class representing a person.

```
#include <zmogus.h>
```

Inheritance diagram for Zmogus:



Public Member Functions

- [Zmogus](#) ()=default
Default constructor.
- [Zmogus](#) (const std::string &[vardas](#), const std::string &[pavarde](#))
Constructor with name and surname.
- [Zmogus](#) (const [Zmogus](#) &other)
Copy constructor.
- [Zmogus](#) & [operator=](#) (const [Zmogus](#) &other)
Copy assignment operator.
- [Zmogus](#) ([Zmogus](#) &&other) noexcept
Move constructor.
- [Zmogus](#) & [operator=](#) ([Zmogus](#) &&other) noexcept
Move assignment operator.
- virtual [~Zmogus](#) ()=0
Virtual destructor Pure virtual destructor makes this class abstract.
- virtual void [print](#) (std::ostream &os) const =0
Print person information to output stream.
- virtual void [read](#) (std::istream &is)=0
Read person information from input stream.
- std::string [vardas](#) () const
Get first name.
- std::string [pavarde](#) () const
Get last name.
- void [setVardas](#) (const std::string &[vardas](#))
Set first name.
- void [setPavarde](#) (const std::string &[pavarde](#))
Set last name.

Protected Attributes

- std::string `vardas_`
First name of the person.
- std::string `pavarde_`
Last name of the person.

4.6.1 Detailed Description

Abstract base class representing a person.

This class serves as an abstract base class for all person types. It implements common attributes (name, surname) and declares pure virtual methods.

4.6.2 Constructor & Destructor Documentation

4.6.2.1 Zmogus() [1/4]

```
Zmogus::Zmogus () [default]
```

Default constructor.

4.6.2.2 Zmogus() [2/4]

```
Zmogus::Zmogus (  
    const std::string & vardas,  
    const std::string & pavarde) [inline]
```

Constructor with name and surname.

Parameters

<code>vardas</code>	First name
<code>pavarde</code>	Last name

4.6.2.3 Zmogus() [3/4]

```
Zmogus::Zmogus (  
    const Zmogus & other) [inline]
```

Copy constructor.

Parameters

<code>other</code>	Another person to copy from
--------------------	-----------------------------

4.6.2.4 Zmogus() [4/4]

```
Zmogus::Zmogus (  
    Zmogus && other) [noexcept]
```

Move constructor.

Parameters

<i>other</i>	Another person to move from
--------------	-----------------------------

4.6.2.5 ~Zmogus()

```
Zmogus::~~Zmogus () [pure virtual]
```

Virtual destructor Pure virtual destructor makes this class abstract.

4.6.3 Member Function Documentation**4.6.3.1 operator=()** [1/2]

```
Zmogus & Zmogus::operator= (
    const Zmogus & other)
```

Copy assignment operator.

Parameters

<i>other</i>	Another person to copy from
--------------	-----------------------------

Returns

Reference to this person after assignment

4.6.3.2 operator=() [2/2]

```
Zmogus & Zmogus::operator= (
    Zmogus && other) [noexcept]
```

Move assignment operator.

Parameters

<i>other</i>	Another person to move from
--------------	-----------------------------

Returns

Reference to this person after assignment

4.6.3.3 pavarde()

```
std::string Zmogus::pavarde () const [inline]
```

Get last name.

Returns

Last name of the person

4.6.3.4 print()

```
virtual void Zmogus::print (
    std::ostream & os) const [pure virtual]
```

Print person information to output stream.

Parameters

<i>os</i>	Output stream to print to
-----------	---------------------------

Pure virtual method that derived classes must implement

Implemented in [Studentas](#).

4.6.3.5 read()

```
virtual void Zmogus::read (  
    std::istream & is) [pure virtual]
```

Read person information from input stream.

Parameters

<i>is</i>	Input stream to read from
-----------	---------------------------

Pure virtual method that derived classes must implement

Implemented in [Studentas](#).

4.6.3.6 setPavarde()

```
void Zmogus::setPavarde (  
    const std::string & pavarde) [inline]
```

Set last name.

Parameters

<i>pavarde</i>	Last name to set
----------------	------------------

4.6.3.7 setVardas()

```
void Zmogus::setVardas (  
    const std::string & vardas) [inline]
```

Set first name.

Parameters

<i>vardas</i>	First name to set
---------------	-------------------

4.6.3.8 vardas()

```
std::string Zmogus::vardas () const [inline]
```

Get first name.

Returns

First name of the person

4.6.4 Member Data Documentation

4.6.4.1 pavarde_

```
std::string Zmogus::pavarde_ [protected]
```

Last name of the person.

4.6.4.2 vardas_

```
std::string Zmogus::vardas_ [protected]
```

First name of the person.

The documentation for this class was generated from the following files:

- [zmogus.h](#)
- [zmogus.cpp](#)

Chapter 5

File Documentation

5.1 funkcijos.cpp File Reference

```
#include "funkcijos.h"
#include "laikas.h"
#include "vector.h"
#include <cstdlib>
#include <ctime>
```

Functions

- bool [compareByVardas](#) (const [Studentas](#) &a, const [Studentas](#) &b)
Compare students by first name.
- bool [compareByPavarde](#) (const [Studentas](#) &a, const [Studentas](#) &b)
Compare students by last name.
- bool [compareByGalutinis](#) (const [Studentas](#) &a, const [Studentas](#) &b)
Compare students by final grade (descending)
- template<typename Container>
void [testuotiDuomenuApdorojimaImpl](#) (const std::string &aplankas, int skaicius)
Implementation of data processing test.
- void [testuotiDuomenuApdorojima](#) (const std::string &aplankas, int skaicius, int testChoice)
Test data processing performance.
- template void [testuotiDuomenuApdorojimaImpl](#)< std::vector< [Studentas](#) > > (const std::string &aplankas, int skaicius)
- template void [testuotiDuomenuApdorojimaImpl](#)< Vector< [Studentas](#) > > (const std::string &aplankas, int skaicius)
- void [testuotiStudentoMetodus](#) ()
Test [Studentas](#) class methods.
- void [testuotiZmogausKlase](#) ()
Test [Zmogus](#) class.

5.1.1 Function Documentation

5.1.1.1 `compareByGalutinis()`

```
bool compareByGalutinis (
    const Studentas & a,
    const Studentas & b)
```

Compare students by final grade (descending)

5.1.1.2 `compareByPavarde()`

```
bool compareByPavarde (
    const Studentas & a,
    const Studentas & b)
```

Compare students by last name.

5.1.1.3 `compareByVardas()`

```
bool compareByVardas (
    const Studentas & a,
    const Studentas & b)
```

Compare students by first name.

5.1.1.4 `testuotiDuomenuApdorojima()`

```
void testuotiDuomenuApdorojima (
    const std::string & aplankas,
    int skaicius,
    int testChoice)
```

Test data processing performance.

Parameters

<i>aplankas</i>	Directory for files
<i>skaicius</i>	Number of students
<i>testChoice</i>	Test choice (2 for <code>std::vector</code> , 3 for custom Vector)

5.1.1.5 `testuotiDuomenuApdorojimaImpl()`

```
template<typename Container>
void testuotiDuomenuApdorojimaImpl (
    const std::string & aplankas,
    int skaicius)
```

Implementation of data processing test.

Parameters

<i>aplankas</i>	Directory for test files
<i>skaicius</i>	Number of students

5.1.1.6 testuotiDuomenuApdorojimaImpl< std::vector< Studentas > >()

```
template void testuotiDuomenuApdorojimaImpl< std::vector< Studentas > > (
    const std::string & aplankas,
    int skaicius)
```

5.1.1.7 testuotiDuomenuApdorojimaImpl< Vector< Studentas > >()

```
template void testuotiDuomenuApdorojimaImpl< Vector< Studentas > > (
    const std::string & aplankas,
    int skaicius)
```

5.1.1.8 testuotiStudentoMetodus()

```
void testuotiStudentoMetodus ()
```

Test [Studentas](#) class methods.

5.1.1.9 testuotiZmogausKlase()

```
void testuotiZmogausKlase ()
```

Test [Zmogus](#) class.

5.2 funkcijos.h File Reference

```
#include <iostream>
#include <string>
#include <vector>
#include <algorithm>
#include <iomanip>
#include <fstream>
#include <sstream>
#include <stdexcept>
#include <execution>
#include <numeric>
#include <cassert>
#include "zmogus.h"
```

Classes

- class [Studentas](#)
Student class that inherits from [Zmogus](#) (Person) abstract class.

Functions

- bool [compareByVardas](#) (const [Studentas](#) &a, const [Studentas](#) &b)
Compare students by first name.
- bool [compareByPavarde](#) (const [Studentas](#) &a, const [Studentas](#) &b)
Compare students by last name.
- bool [compareByGalutinis](#) (const [Studentas](#) &a, const [Studentas](#) &b)
Compare students by final grade (descending)
- template<typename Container>
void [skaitytiIsFailo](#) (Container &grupe, const std::string &failoPavadinimas)
Read students from file into container.
- template<typename Container>
void [sortStudentai](#) (Container &grupe, char sortingOption)
Sort students in container.
- template<typename Container>
void [skirstytiStudentus](#) (Container &grupe, Container &kietiakiai, Container &vargsai)
Separate students into two groups based on final grade.
- template<typename Container>
void [isvestiStudentusIFaila](#) (const Container &studentai, const std::string &failoPavadinimas, char ats)
Output students to file.
- template<typename Container>
void [generuotiStudentus](#) (Container &grupe, int kiekis)
Generate random students for testing.
- template<typename Container>
void [testuotiDuomenuApdorojimaImpl](#) (const std::string &aplankas, int skaicius)
Implementation of data processing test.
- void [testuotiDuomenuApdorojima](#) (const std::string &aplankas, int skaicius, int testChoice)
Test data processing performance.
- void [testuotiStudentoMetodus](#) ()
Test [Studentas](#) class methods.
- void [testuotiZmogausKlase](#) ()
Test [Zmogus](#) class.

5.2.1 Function Documentation

5.2.1.1 compareByGalutinis()

```
bool compareByGalutinis (
    const Studentas & a,
    const Studentas & b)
```

Compare students by final grade (descending)

5.2.1.2 compareByPavarde()

```
bool compareByPavarde (  
    const Studentas & a,  
    const Studentas & b)
```

Compare students by last name.

5.2.1.3 compareByVardas()

```
bool compareByVardas (  
    const Studentas & a,  
    const Studentas & b)
```

Compare students by first name.

5.2.1.4 generuotiStudentus()

```
template<typename Container>  
void generuotiStudentus (  
    Container & grupe,  
    int kiekis)
```

Generate random students for testing.

Parameters

<i>grupe</i>	Container to store generated students
<i>kiekis</i>	Number of students to generate

5.2.1.5 isvestiStudentusIFaila()

```
template<typename Container>  
void isvestiStudentusIFaila (  
    const Container & studentai,  
    const std::string & failoPavadinimas,  
    char ats)
```

Output students to file.

Parameters

<i>studentai</i>	Container of students
<i>failoPavadinimas</i>	Name of output file
<i>ats</i>	Answer type ('v' - average, 'm' - median)

5.2.1.6 skaitytiIsFailo()

```
template<typename Container>  
void skaitytiIsFailo (  
    Container & grupe,  
    const std::string & failoPavadinimas)
```

Read students from file into container.

Parameters

<i>grupe</i>	Container to store students
<i>failoPavadinimas</i>	Name of the file

5.2.1.7 skirstytiStudentus()

```
template<typename Container>
void skirstytiStudentus (
    Container & grupe,
    Container & kietiakiiai,
    Container & vargsai)
```

Separate students into two groups based on final grade.

Parameters

<i>grupe</i>	Original container of students
<i>kietiakiiai</i>	Container for students with grade $\geq$ 5.0
<i>vargsai</i>	Container for students with grade $<$ 5.0

5.2.1.8 sortStudentai()

```
template<typename Container>
void sortStudentai (
    Container & grupe,
    char sortingOption)
```

Sort students in container.

Parameters

<i>grupe</i>	Container of students
<i>sortingOption</i>	Sorting option ('v' - by name, 'p' - by surname, 'g' - by grade)

5.2.1.9 testuotiDuomenuApdorojima()

```
void testuotiDuomenuApdorojima (
    const std::string & aplankas,
    int skaicius,
    int testChoice)
```

Test data processing performance.

Parameters

<i>aplankas</i>	Directory for files
<i>skaicius</i>	Number of students
<i>testChoice</i>	Test choice (2 for std::vector, 3 for custom Vector)

5.2.1.10 testuotiDuomenuApdorojimaImpl()

```
template<typename Container>
void testuotiDuomenuApdorojimaImpl (
    const std::string & aplankas,
    int skaicius)
```

Implementation of data processing test.

Parameters

<i>aplangas</i>	Directory for test files
<i>skaicius</i>	Number of students

5.2.1.11 testuotiStudentoMetodus()

```
void testuotiStudentoMetodus ()
```

Test [Studentas](#) class methods.

5.2.1.12 testuotiZmogausKlase()

```
void testuotiZmogausKlase ()
```

Test [Zmogus](#) class.

5.3 funkcijos.h

[Go to the documentation of this file.](#)

```
00001 #ifndef FUNKCIJOS_H
00002 #define FUNKCIJOS_H
00003
00004 #include <iostream>
00005 #include <string>
00006 #include <vector>
00007 #include <algorithm>
00008 #include <iomanip>
00009 #include <fstream>
00010 #include <sstream>
00011 #include <stdexcept>
00012 #include <execution>
00013 #include <numeric>
00014 #include <cassert>
00015 #include "zmogus.h"
00016
00024 class Studentas : public Zmogus
00025 {
00026 // realizacija
00027 private:
00028     std::vector<int> nd_;
00029     int egzaminas_;
00030     double galutinis_;
00031
00032 // interfeisas
00033 public:
00035     static int destruktoriuSk;
00036
00041     Studentas() : Zmogus(), egzaminas_(0), galutinis_(0) { }
00042
00047     Studentas(std::istream& is);
00048
```

```

00054     Studentas(const std::string& vardas, const std::string& pavarde)
00055         : Zmogus(vardas, pavarde), egzaminas_(0), galutinis_(0) { }
00056
00057     // Rule of Five
00062     Studentas(const Studentas& other);
00063
00069     Studentas& operator=(const Studentas& other);
00070
00075     Studentas(Studentas&& other) noexcept;
00076
00082     Studentas& operator=(Studentas&& other) noexcept;
00083
00088     ~Studentas() override;
00089
00090     // Implementation of pure virtual methods
00095     void print(std::ostream& os) const override;
00096
00101     void read(std::istream& is) override;
00102
00103     // Getteriai
00108     inline std::vector<int> nd() const { return nd_; }
00109
00114     inline int egzaminas() const { return egzaminas_; }
00115
00120     inline double galutinis() const { return galutinis_; }
00121
00122     // Setteriai
00127     inline void setEgzaminas(int egzaminas) { egzaminas_ = egzaminas; }
00128
00133     inline void setGalutinis(double galutinis) { galutinis_ = galutinis; }
00134
00135     // Metodai
00141     std::istream& readStudent(std::istream& is);
00142
00148     void addND(int pazymys);
00149
00153     void clearND();
00154
00159     double galBalas() const {
00160         if (nd_.empty()) return egzaminas_;
00161         double ndVidurkis = skaiciuotiVid(nd_);
00162         return 0.4 * ndVidurkis + 0.6 * egzaminas_;
00163     }
00164
00169     double galBalasMed() const {
00170         if (nd_.empty()) return egzaminas_;
00171         double ndMediana = skaiciuotiMed(nd_);
00172         return 0.4 * ndMediana + 0.6 * egzaminas_;
00173     }
00174
00180     template <typename Container>
00181     static double skaiciuotiVid(const Container& nd) {
00182         if (nd.empty()) {
00183             throw std::runtime_error("Namu darbu sarasas negali buti tuscias");
00184         }
00185         double suma = 0.0;
00186         for (const auto& elem : nd) {
00187             suma += elem;
00188         }
00189         return suma / nd.size();
00190     }
00191
00197     template <typename Container>
00198     static double skaiciuotiMed(Container nd) {
00199         if (nd.empty()) {
00200             throw std::runtime_error("Namu darbu sarasas negali buti tuscias");
00201         }
00202         std::sort(nd.begin(), nd.end());
00203         size_t dydis = nd.size();
00204         if (dydis % 2 == 0) {
00205             return (nd[dydis / 2 - 1] + nd[dydis / 2]) / 2.0;
00206         } else {
00207             return nd[dydis / 2];
00208         }
00209     }
00210
00216     template<typename Container>
00217     static void nuskaitymasFile(Container& grupe, const std::string& failoPavadinimas) {
00218         std::ifstream failas(failoPavadinimas);
00219         if (!failas.is_open()) {
00220             throw std::runtime_error("Nepavyko atidaryti failo: " + failoPavadinimas);
00221         }
00222
00223         std::string eilute;
00224
00225         // Praleisti pirmą eilutę (header)
00226         if (std::getline(failas, eilute)) {

```

```

00227         // Pirma eilutė praleista
00228     }
00229
00230     // Dabar skaityti studentų duomenis
00231     while (std::getline(failas, eilute)) {
00232         if (eilute.empty()) continue;
00233
00234         std::istringstream iss(eilute);
00235         Studentas studentas;
00236         try {
00237             studentas.readStudent(iss);
00238             studentas.setGalutinis(studentas.galBalas());
00239             grupe.push_back(studentas);
00240         } catch (const std::exception& e) {
00241             std::cerr << "Klaida skaitant studenta: " << e.what() << std::endl;
00242         }
00243     }
00244     failas.close();
00245 }
00246 };
00247
00248 // Comparison functions
00252 bool compareByVardas(const Studentas& a, const Studentas& b);
00253
00257 bool compareByPavarde(const Studentas& a, const Studentas& b);
00258
00262 bool compareByGalutinis(const Studentas& a, const Studentas& b);
00263
00264 // Template functions for container operations
00270 template<typename Container>
00271 void skaitytiIsFailo(Container& grupe, const std::string& failoPavadinimas) {
00272     Studentas::nuskaitymasFile(grupe, failoPavadinimas);
00273 }
00274
00280 template<typename Container>
00281 void sortStudentai(Container& grupe, char sortingOption) {
00282     switch (tolower(sortingOption)) {
00283         case 'v':
00284             std::sort(grupe.begin(), grupe.end(), compareByVardas);
00285             break;
00286         case 'p':
00287             std::sort(grupe.begin(), grupe.end(), compareByPavarde);
00288             break;
00289         case 'g':
00290             std::sort(grupe.begin(), grupe.end(), compareByGalutinis);
00291             break;
00292         default:
00293             std::sort(grupe.begin(), grupe.end(), compareByPavarde);
00294             break;
00295     }
00296 }
00297
00304 template<typename Container>
00305 void skirstytiStudentus(Container& grupe, Container& kietiakai, Container& vargsai) {
00306     // Using std::partition for efficiency
00307     auto partition_point = std::partition(grupe.begin(), grupe.end(),
00308         [](const Studentas& s) { return s.galutinis() >= 5.0; });
00309
00310     // Copy to respective containers
00311     kietiakai.assign(grupe.begin(), partition_point);
00312     vargsai.assign(partition_point, grupe.end());
00313
00314     // Clear original container
00315     grupe.clear();
00316 }
00317
00324 template<typename Container>
00325 void ivestiStudentusIFaila(const Container& studentai, const std::string& failoPavadinimas, char ats)
00326 {
00327     std::ofstream failas(failoPavadinimas);
00328     if (!failas.is_open()) {
00329         throw std::runtime_error("Nepavyko sukurti failo: " + failoPavadinimas);
00330     }
00331
00332     // Header
00333     failas << std::left << std::setw(15) << "Vardas"
00334             << std::setw(15) << "Pavarde"
00335             << std::setw(20) << "Galutinis (" << (tolower(ats) == 'v' ? "Vid." : "Med.") << ")"
00336             << std::endl;
00337     failas << std::string(50, '-') << std::endl;
00338
00339     // Student data
00340     for (const auto& studentas : studentai) {
00341         failas << std::left << std::setw(15) << studentas.vardas()
00342             << std::setw(15) << studentas.pavarde()
00343             << std::fixed << std::setprecision(2)
00344             << (tolower(ats) == 'v' ? studentas.galBalas() : studentas.galBalasMed())

```

```

00344         « std::endl;
00345     }
00346
00347     failas.close();
00348 }
00349
00355 template<typename Container>
00356 void generuotiStudentus(Container& grupe, int kiekis) {
00357     grupe.clear();
00358     grupe.reserve(kiekis); // Reserve space if container supports it
00359
00360     for (int i = 1; i <= kiekis; ++i) {
00361         Studentas studentas("Vardas" + std::to_string(i), "Pavarde" + std::to_string(i));
00362
00363         // Generate 5-10 random homework grades
00364         int ndKiekis = 5 + rand() % 6;
00365         for (int j = 0; j < ndKiekis; ++j) {
00366             studentas.addND(1 + rand() % 10);
00367         }
00368
00369         // Generate exam grade
00370         studentas.setEgzaminas(1 + rand() % 10);
00371         studentas.setGalutinis(studentas.galBalas());
00372
00373         grupe.push_back(studentas);
00374     }
00375 }
00376
00382 template<typename Container>
00383 void testuotiDuomenuApdorojimaImpl(const std::string& aplankas, int skaicius);
00384
00391 void testuotiDuomenuApdorojima(const std::string& aplankas, int skaicius, int testChoice);
00392
00396 void testuotiStudentoMetodus();
00397
00401 void testuotiZmogausKlase();
00402
00403 #endif // FUNKCIJOS_H

```

5.4 laikas.cpp File Reference

```

#include "Laikas.h"
#include "funkcijos.h"

```

5.5 laikas.h File Reference

```

#include <chrono>
#include <iostream>

```

Classes

- class [Laikas](#)

Class for measuring execution time.

5.6 laikas.h

[Go to the documentation of this file.](#)

```

00001 #ifndef LAIKAS_H
00002 #define LAIKAS_H
00003

```



```
00004 #include <chrono>
00005 #include <iostream>
00006
00014 class Laikas
00015 {
00016 private:
00017     std::chrono::high_resolution_clock::time_point start;
00018     std::chrono::high_resolution_clock::time_point end;
00019     std::string veiksmoPavadinimas;
00020
00021 public:
00026     Laikas(const std::string& pavadinimas);
00027
00032     void pradeti();
00033
00038     void baigti();
00039
00044     double gautiLaikoSkirtuma();
00045 };
00046
00047 #endif
```

5.7 main.cpp File Reference

```
#include "funkcijos.h"
#include "laikas.h"
#include "zmogus.h"
#include "vector.h"
```

Functions

- int [main](#) ()

5.7.1 Function Documentation

5.7.1.1 main()

```
int main ()
```

5.8 testai.cpp File Reference

```
#include <gtest/gtest.h>
#include "funkcijos.h"
#include "zmogus.h"
```

Classes

- class [StudentasTest](#)

Functions

- [TEST_F \(StudentasTest, DefaultConstructor\)](#)
- [TEST_F \(StudentasTest, CopyConstructor\)](#)
- [TEST_F \(StudentasTest, CopyAssignmentOperator\)](#)
- [TEST_F \(StudentasTest, MoveConstructor\)](#)
- [TEST_F \(StudentasTest, MoveAssignmentOperator\)](#)
- [TEST_F \(StudentasTest, Destructor\)](#)
- [TEST_F \(StudentasTest, GradeCalculation\)](#)
- [TEST_F \(StudentasTest, InputOutput\)](#)
- [int main](#) (int argc, char **argv)

5.8.1 Function Documentation

5.8.1.1 main()

```
int main (  
    int argc,  
    char ** argv)
```

5.8.1.2 TEST_F() [1/8]

```
TEST_F (  
    StudentasTest ,  
    CopyAssignmentOperator )
```

5.8.1.3 TEST_F() [2/8]

```
TEST_F (  
    StudentasTest ,  
    CopyConstructor )
```

5.8.1.4 TEST_F() [3/8]

```
TEST_F (  
    StudentasTest ,  
    DefaultConstructor )
```

5.8.1.5 TEST_F() [4/8]

```
TEST_F (  
    StudentasTest ,  
    Destructor )
```

5.8.1.6 TEST_F() [5/8]

```
TEST_F (
    StudentasTest ,
    GradeCalculation )
```

5.8.1.7 TEST_F() [6/8]

```
TEST_F (
    StudentasTest ,
    InputOutput )
```

5.8.1.8 TEST_F() [7/8]

```
TEST_F (
    StudentasTest ,
    MoveAssignmentOperator )
```

5.8.1.9 TEST_F() [8/8]

```
TEST_F (
    StudentasTest ,
    MoveConstructor )
```

5.9 vector.h File Reference

```
#include <algorithm>
#include <cstdlib>
#include <initializer_list>
#include <stdexcept>
#include <utility>
#include <iterator>
#include <memory>
```

Classes

- class [Vector< T >](#)

Dinaminis šabloninis konteineris, panašus į std::vector.

5.10 vector.h

[Go to the documentation of this file.](#)

```

00001 #ifndef VECTOR_H
00002 #define VECTOR_H
00003
00004 #include <algorithm>
00005 #include <cstddef>
00006 #include <initializer_list>
00007 #include <stdexcept>
00008 #include <utility>
00009 #include <iterator>
00010 #include <memory>
00011
00017 template <typename T>
00018 class Vector {
00019 public:
00020     using value_type = T;
00021     using size_type = std::size_t;
00022     using difference_type = std::ptrdiff_t;
00023     using reference = T&;
00024     using const_reference = const T&;
00025     using pointer = T*;
00026     using const_pointer = const T*;
00027     using iterator = T*;
00028     using const_iterator = const T*;
00029     using reverse_iterator = std::reverse_iterator<iterator>;
00030     using const_reverse_iterator = std::reverse_iterator<const_iterator>;
00031
00032 private:
00033     pointer data_ = nullptr;
00034     size_type size_ = 0;
00035     size_type capacity_ = 0;
00036     size_t realloc_count_ = 0;
00037
00043     void reallocate(size_type new_capacity);
00044
00048     void destroy_elements();
00049
00050 public:
00054     Vector() = default;
00055
00062     explicit Vector(size_type count, const T& value = T());
00063
00069     Vector(std::initializer_list<T> init);
00070
00076     Vector(const Vector& other);
00077
00083     Vector(Vector&& other) noexcept;
00084
00088     ~Vector();
00089
00090     // Operatoriai
00091     Vector& operator=(const Vector& other);
00092     Vector& operator=(Vector&& other) noexcept;
00093     Vector& operator=(std::initializer_list<T> ilist);
00094
00095     // Priega prie elementu
00102     reference at(size_type pos);
00103
00110     const_reference at(size_type pos) const;
00111
00112     reference operator[](size_type pos);
00113     const_reference operator[](size_type pos) const;
00114     reference front();
00115     const_reference front() const;
00116     reference back();
00117     const_reference back() const;
00118     pointer data() noexcept;
00119     const_pointer data() const noexcept;
00120
00121     // Iteratoriai
00122     iterator begin() noexcept;
00123     const_iterator begin() const noexcept;
00124     const_iterator cbegin() const noexcept;
00125     iterator end() noexcept;
00126     const_iterator end() const noexcept;
00127     const_iterator cend() const noexcept;
00128     reverse_iterator rbegin() noexcept;
00129     const_reverse_iterator rbegin() const noexcept;
00130     const_reverse_iterator crbegin() const noexcept;
00131     reverse_iterator rend() noexcept;
00132     const_reverse_iterator rend() const noexcept;
00133     const_reverse_iterator crend() const noexcept;
00134

```

```

00135     // Naudingos funkcijos
00136     bool empty() const noexcept;
00137     size_type size() const noexcept;
00138     size_type capacity() const noexcept;
00139     void reserve(size_type new_cap);
00140     void shrink_to_fit();
00141     void clear() noexcept;
00142
00143     // Modifikavimas
00144     iterator insert(const_iterator pos, const T& value);
00145     iterator insert(const_iterator pos, T&& value);
00146     iterator insert(const_iterator pos, size_type count, const T& value);
00147     iterator erase(const_iterator pos);
00148     iterator erase(const_iterator first, const_iterator last);
00149     void push_back(const T& value);
00150     void push_back(T&& value);
00151
00152     template <class... Args>
00153     reference emplace_back(Args&&... args);
00154
00155     void pop_back();
00156     void resize(size_type count);
00157     void resize(size_type count, const T& value);
00158     void swap(Vector& other) noexcept;
00159
00160     // Naujos funkcijos
00161     void assign(size_type count, const T& value);
00162     template<class InputIt>
00163     void assign(InputIt first, InputIt last);
00164     void assign(std::initializer_list<T> ilist);
00165
00166     size_t reallocations() const;
00167
00168     // Lyginimo operatoriai
00169     friend bool operator==(const Vector& lhs, const Vector& rhs) {
00170         return lhs.size_ == rhs.size_ && std::equal(lhs.data_, lhs.data_ + lhs.size_, rhs.data_);
00171     }
00172     friend bool operator!=(const Vector& lhs, const Vector& rhs) {
00173         return !(lhs == rhs);
00174     }
00175 };
00176
00177 #endif // VECTOR_H

```

5.11 vector_testai.cpp File Reference

```

#include <gtest/gtest.h>
#include "vector.h"
#include <vector>
#include <string>
#include <algorithm>
#include <stdexcept>

```

Classes

- class [VectorTest](#)

Functions

- [TEST_F](#) ([VectorTest](#), DefaultConstructor)
- [TEST_F](#) ([VectorTest](#), CountValueConstructor)
- [TEST_F](#) ([VectorTest](#), InitializerListConstructor)
- [TEST_F](#) ([VectorTest](#), CopyConstructor)
- [TEST_F](#) ([VectorTest](#), MoveConstructor)
- [TEST_F](#) ([VectorTest](#), CopyAssignment)

- [TEST_F \(VectorTest, MoveAssignment\)](#)
- [TEST_F \(VectorTest, InitializerListAssignment\)](#)
- [TEST_F \(VectorTest, ElementAccess\)](#)
- [TEST_F \(VectorTest, CapacityOperations\)](#)
- [TEST_F \(VectorTest, PushBackAndPopBack\)](#)
- [TEST_F \(VectorTest, PushBackMoveSemantics\)](#)
- [TEST_F \(VectorTest, EmplaceBack\)](#)
- [TEST_F \(VectorTest, Clear\)](#)
- [TEST_F \(VectorTest, Resize\)](#)
- [TEST_F \(VectorTest, InsertSingleElement\)](#)
- [TEST_F \(VectorTest, InsertMoveElement\)](#)
- [TEST_F \(VectorTest, InsertMultipleElements\)](#)
- [TEST_F \(VectorTest, EraseSingleElement\)](#)
- [TEST_F \(VectorTest, EraseRange\)](#)
- [TEST_F \(VectorTest, Iterators\)](#)
- [TEST_F \(VectorTest, ComparisonOperators\)](#)
- [TEST_F \(VectorTest, AlgorithmCompatibility\)](#)
- [TEST_F \(VectorTest, EdgeCases\)](#)
- [TEST_F \(VectorTest, ReserveEfficiency\)](#)
- [int main \(int argc, char **argv\)](#)

5.11.1 Function Documentation

5.11.1.1 main()

```
int main (
    int argc,
    char ** argv)
```

5.11.1.2 TEST_F() [1/25]

```
TEST_F (
    VectorTest ,
    AlgorithmCompatibility )
```

5.11.1.3 TEST_F() [2/25]

```
TEST_F (
    VectorTest ,
    CapacityOperations )
```

5.11.1.4 TEST_F() [3/25]

```
TEST_F (
    VectorTest ,
    Clear )
```

5.11.1.5 TEST_F() [4/25]

```
TEST_F (
    VectorTest ,
    ComparisonOperators )
```

5.11.1.6 TEST_F() [5/25]

```
TEST_F (
    VectorTest ,
    CopyAssignment )
```

5.11.1.7 TEST_F() [6/25]

```
TEST_F (
    VectorTest ,
    CopyConstructor )
```

5.11.1.8 TEST_F() [7/25]

```
TEST_F (
    VectorTest ,
    CountValueConstructor )
```

5.11.1.9 TEST_F() [8/25]

```
TEST_F (
    VectorTest ,
    DefaultConstructor )
```

5.11.1.10 TEST_F() [9/25]

```
TEST_F (
    VectorTest ,
    EdgeCases )
```

5.11.1.11 TEST_F() [10/25]

```
TEST_F (
    VectorTest ,
    ElementAccess )
```

5.11.1.12 TEST_F() [11/25]

```
TEST_F (
    VectorTest ,
    EmplaceBack )
```

5.11.1.13 TEST_F() [12/25]

```
TEST_F (
    VectorTest ,
    EraseRange )
```

5.11.1.14 TEST_F() [13/25]

```
TEST_F (
    VectorTest ,
    EraseSingleElement )
```

5.11.1.15 TEST_F() [14/25]

```
TEST_F (
    VectorTest ,
    InitializerListAssignment )
```

5.11.1.16 TEST_F() [15/25]

```
TEST_F (
    VectorTest ,
    InitializerListConstructor )
```

5.11.1.17 TEST_F() [16/25]

```
TEST_F (
    VectorTest ,
    InsertMoveElement )
```

5.11.1.18 TEST_F() [17/25]

```
TEST_F (
    VectorTest ,
    InsertMultipleElements )
```

5.11.1.19 TEST_F() [18/25]

```
TEST_F (
    VectorTest ,
    InsertSingleElement )
```

5.11.1.20 TEST_F() [19/25]

```
TEST_F (
    VectorTest ,
    Iterators )
```


5.11.1.21 TEST_F() [20/25]

```
TEST_F (
    VectorTest ,
    MoveAssignment )
```

5.11.1.22 TEST_F() [21/25]

```
TEST_F (
    VectorTest ,
    MoveConstructor )
```

5.11.1.23 TEST_F() [22/25]

```
TEST_F (
    VectorTest ,
    PushBackAndPopBack )
```

5.11.1.24 TEST_F() [23/25]

```
TEST_F (
    VectorTest ,
    PushBackMoveSemantics )
```

5.11.1.25 TEST_F() [24/25]

```
TEST_F (
    VectorTest ,
    ReserveEfficiency )
```

5.11.1.26 TEST_F() [25/25]

```
TEST_F (
    VectorTest ,
    Resize )
```

5.12 zmogus.cpp File Reference

```
#include "zmogus.h"
```

Functions

- `std::ostream & operator<<` (`std::ostream &os`, `const Zmogus &zmogus`)
Output operator for Zmogus class.
- `std::istream & operator>>` (`std::istream &is`, `Zmogus &zmogus`)
Input operator for Zmogus class.

5.12.1 Function Documentation

5.12.1.1 `operator<<()`

```
std::ostream & operator<< (  
    std::ostream & os,  
    const Zmogus & zmogus)
```

Output operator for [Zmogus](#) class.

Parameters

<i>os</i>	Output stream
<i>zmogus</i>	Person to output

Returns

Reference to output stream

5.12.1.2 `operator>>()`

```
std::istream & operator>> (  
    std::istream & is,  
    Zmogus & zmogus)
```

Input operator for [Zmogus](#) class.

Parameters

<i>is</i>	Input stream
<i>zmogus</i>	Person to input to

Returns

Reference to input stream

5.13 zmogus.h File Reference

```
#include <string>  
#include <iostream>
```

Classes

- class [Zmogus](#)

Abstract base class representing a person.

Functions

- `std::ostream & operator<< (std::ostream &os, const Zmogus &zmogus)`
Output operator for Zmogus class.
- `std::istream & operator>> (std::istream &is, Zmogus &zmogus)`
Input operator for Zmogus class.

5.13.1 Function Documentation

5.13.1.1 operator<<()

```
std::ostream & operator<< (  
    std::ostream & os,  
    const Zmogus & zmogus)
```

Output operator for Zmogus class.

Parameters

<i>os</i>	Output stream
<i>zmogus</i>	Person to output

Returns

Reference to output stream

5.13.1.2 operator>>()

```
std::istream & operator>> (  
    std::istream & is,  
    Zmogus & zmogus)
```

Input operator for Zmogus class.

Parameters

<i>is</i>	Input stream
<i>zmogus</i>	Person to input to

Returns

Reference to input stream

5.14 zmogus.h

[Go to the documentation of this file.](#)

```

00001 #ifndef ZMOGUS_H
00002 #define ZMOGUS_H
00003
00004 #include <string>
00005 #include <iostream>
00006
00014 class Zmogus
00015 {
00016 protected:
00017     std::string vardas_;
00018     std::string pavarde_;
00019
00020 public:
00024     Zmogus() = default;
00025
00031     Zmogus(const std::string& vardas, const std::string& pavarde)
00032         : vardas_(vardas), pavarde_(pavarde) {}
00033
00034     // Rule of Five
00039     Zmogus(const Zmogus& other) : vardas_(other.vardas_), pavarde_(other.pavarde_) {}
00040
00046     Zmogus& operator=(const Zmogus& other);
00047
00052     Zmogus(Zmogus&& other) noexcept;
00053
00059     Zmogus& operator=(Zmogus&& other) noexcept;
00060
00065     virtual ~Zmogus() = 0;
00066
00073     virtual void print(std::ostream& os) const = 0;
00074
00081     virtual void read(std::istream& is) = 0;
00082
00083     // Getters and setters
00088     inline std::string vardas() const { return vardas_; }
00089
00094     inline std::string pavarde() const { return pavarde_; }
00095
00100     inline void setVardas(const std::string& vardas) { vardas_ = vardas; }
00101
00106     inline void setPavarde(const std::string& pavarde) { pavarde_ = pavarde; }
00107 };
00108
00115 std::ostream& operator<<(std::ostream& os, const Zmogus& zmogus);
00116
00123 std::istream& operator>>(std::istream& is, Zmogus& zmogus);
00124
00125 #endif // ZMOGUS_H

```

Index

~Studentas
 Studentas, [12](#)
~Vector
 Vector< T >, [24](#)
~Zmogus
 Zmogus, [36](#)

addND
 Studentas, [12](#)
assign
 Vector< T >, [24](#)
at
 Vector< T >, [25](#)

back
 Vector< T >, [25](#)
baigti
 Laikas, [8](#)
begin
 Vector< T >, [26](#)

capacity
 Vector< T >, [26](#)
capacity_
 Vector< T >, [32](#)
cbegin
 Vector< T >, [26](#)
cend
 Vector< T >, [26](#)
clear
 Vector< T >, [26](#)
clearND
 Studentas, [13](#)
compareByGalutinis
 funkcijos.cpp, [40](#)
 funkcijos.h, [42](#)
compareByPavarde
 funkcijos.cpp, [40](#)
 funkcijos.h, [42](#)
compareByVardas
 funkcijos.cpp, [40](#)
 funkcijos.h, [43](#)
const_iterator
 Vector< T >, [21](#)
const_pointer
 Vector< T >, [21](#)
const_reference
 Vector< T >, [21](#)
const_reverse_iterator
 Vector< T >, [21](#)

crbegin
 Vector< T >, [26](#)
crend
 Vector< T >, [26](#)

data
 Vector< T >, [27](#)
data_
 Vector< T >, [32](#)
destroy_elements
 Vector< T >, [27](#)
destruktoriuSk
 Studentas, [17](#)
difference_type
 Vector< T >, [22](#)

egzaminas
 Studentas, [13](#)
egzaminas_
 Studentas, [17](#)
emplace_back
 Vector< T >, [27](#)
empty
 Vector< T >, [27](#)
end
 Laikas, [8](#)
 Vector< T >, [28](#)
erase
 Vector< T >, [28](#)
expectEqual
 VectorTest, [33](#)

front
 Vector< T >, [28](#)
funkcijos.cpp, [39](#)
 compareByGalutinis, [40](#)
 compareByPavarde, [40](#)
 compareByVardas, [40](#)
 testuotiDuomenuApdorojima, [40](#)
 testuotiDuomenuApdorojimaImpl, [40](#)
 testuotiDuomenuApdorojimaImpl< std::vector< Studentas > >, [41](#)
 testuotiDuomenuApdorojimaImpl< Vector< Studentas > >, [41](#)
 testuotiStudentoMetodus, [41](#)
 testuotiZmogausKlase, [41](#)
funkcijos.h, [41](#)
 compareByGalutinis, [42](#)
 compareByPavarde, [42](#)
 compareByVardas, [43](#)

- generuotiStudentus, [43](#)
 - isvestiStudentusIFaila, [43](#)
 - skaitytisFailo, [43](#)
 - skirstytiStudentus, [44](#)
 - sortStudentai, [44](#)
 - testuotiDuomenuApdorojima, [44](#)
 - testuotiDuomenuApdorojimaImpl, [44](#)
 - testuotiStudentoMetodus, [45](#)
 - testuotiZmogausKlase, [45](#)
- galBalas
 - Studentas, [13](#)
- galBalasMed
 - Studentas, [13](#)
- galutinis
 - Studentas, [13](#)
- galutinis_
 - Studentas, [17](#)
- gautiLaikoSkirtuma
 - Laikas, [8](#)
- generuotiStudentus
 - funkcijos.h, [43](#)
- insert
 - Vector< T >, [28](#), [29](#)
- isvestiStudentusIFaila
 - funkcijos.h, [43](#)
- iterator
 - Vector< T >, [22](#)
- Laikas, [7](#)
 - baigti, [8](#)
 - end, [8](#)
 - gautiLaikoSkirtuma, [8](#)
 - Laikas, [7](#)
 - pradeti, [8](#)
 - start, [8](#)
 - veiksmaPavadinimas, [8](#)
- laikas.cpp, [48](#)
- laikas.h, [48](#)
- main
 - main.cpp, [49](#)
 - testai.cpp, [50](#)
 - vector_testai.cpp, [54](#)
- main.cpp, [49](#)
 - main, [49](#)
- nd
 - Studentas, [14](#)
- nd_
 - Studentas, [17](#)
- nuskaitymasFile
 - Studentas, [14](#)
- operator!=
 - Vector< T >, [32](#)
- operator<<
 - zmogus.cpp, [58](#)
 - zmogus.h, [59](#)
- operator>>
 - zmogus.cpp, [58](#)
 - zmogus.h, [59](#)
- operator=
 - Studentas, [14](#)
 - Vector< T >, [29](#)
 - Zmogus, [36](#)
- operator==
 - Vector< T >, [32](#)
- operator[]
 - Vector< T >, [29](#), [30](#)
- pavarde
 - Zmogus, [36](#)
- pavarde_
 - Zmogus, [38](#)
- pointer
 - Vector< T >, [22](#)
- pop_back
 - Vector< T >, [30](#)
- pradeti
 - Laikas, [8](#)
- print
 - Studentas, [15](#)
 - Zmogus, [36](#)
- push_back
 - Vector< T >, [30](#)
- rbegin
 - Vector< T >, [30](#)
- read
 - Studentas, [15](#)
 - Zmogus, [37](#)
- readStudent
 - Studentas, [15](#)
- realloc_count_
 - Vector< T >, [33](#)
- reallocate
 - Vector< T >, [30](#)
- reallocations
 - Vector< T >, [31](#)
- reference
 - Vector< T >, [22](#)
- rend
 - Vector< T >, [31](#)
- reserve
 - Vector< T >, [31](#)
- resize
 - Vector< T >, [31](#)
- reverse_iterator
 - Vector< T >, [22](#)
- setEgzaminas
 - Studentas, [15](#)
- setGalutinis
 - Studentas, [16](#)
- setPavarde
 - Zmogus, [37](#)
- SetUp

- StudentasTest, 18
- VectorTest, 33
- setVardas
 - Zmogus, 37
- shrink_to_fit
 - Vector< T >, 31
- size
 - Vector< T >, 32
- size_
 - Vector< T >, 33
- size_type
 - Vector< T >, 22
- skaiciuotiMed
 - Studentas, 16
- skaiciuotiVid
 - Studentas, 16
- skaitytisFailo
 - funkcijos.h, 43
- skirstytiStudentus
 - funkcijos.h, 44
- sortStudentai
 - funkcijos.h, 44
- start
 - Laikas, 8
- Studentas, 9
 - ~Studentas, 12
 - addND, 12
 - clearND, 13
 - destruktoriuSk, 17
 - egzaminas, 13
 - egzaminas_, 17
 - galBalas, 13
 - galBalasMed, 13
 - galutinis, 13
 - galutinis_, 17
 - nd, 14
 - nd_, 17
 - nuskaitymasFile, 14
 - operator=, 14
 - print, 15
 - read, 15
 - readStudent, 15
 - setEgzaminas, 15
 - setGalutinis, 16
 - skaiciuotiMed, 16
 - skaiciuotiVid, 16
 - Studentas, 11, 12
- StudentasTest, 17
 - SetUp, 18
 - testStudent, 18
- swap
 - Vector< T >, 32
- TearDown
 - VectorTest, 34
- TEST_F
 - testai.cpp, 50, 51
 - vector_testai.cpp, 54–57
- testai.cpp, 49
 - main, 50
 - TEST_F, 50, 51
- testStudent
 - StudentasTest, 18
- testuotiDuomenuApdorojima
 - funkcijos.cpp, 40
 - funkcijos.h, 44
- testuotiDuomenuApdorojimaImpl
 - funkcijos.cpp, 40
 - funkcijos.h, 44
- testuotiDuomenuApdorojimaImpl< std::vector< Studentas > >
 - funkcijos.cpp, 41
- testuotiDuomenuApdorojimaImpl< Vector< Studentas > >
 - funkcijos.cpp, 41
- testuotiStudentoMetodus
 - funkcijos.cpp, 41
 - funkcijos.h, 45
- testuotiZmogausKlase
 - funkcijos.cpp, 41
 - funkcijos.h, 45
- value_type
 - Vector< T >, 22
- vardas
 - Zmogus, 37
- vardas_
 - Zmogus, 38
- Vector
 - Vector< T >, 23, 24
- Vector< T >, 18
 - ~Vector, 24
 - assign, 24
 - at, 25
 - back, 25
 - begin, 26
 - capacity, 26
 - capacity_, 32
 - cbegin, 26
 - cend, 26
 - clear, 26
 - const_iterator, 21
 - const_pointer, 21
 - const_reference, 21
 - const_reverse_iterator, 21
 - crbegin, 26
 - crend, 26
 - data, 27
 - data_, 32
 - destroy_elements, 27
 - difference_type, 22
 - emplace_back, 27
 - empty, 27
 - end, 28
 - erase, 28
 - front, 28
 - insert, 28, 29
 - iterator, 22

- operator!=, [32](#)
- operator=, [29](#)
- operator==, [32](#)
- operator[], [29](#), [30](#)
- pointer, [22](#)
- pop_back, [30](#)
- push_back, [30](#)
- rbegin, [30](#)
- realloc_count_, [33](#)
- reallocate, [30](#)
- reallocations, [31](#)
- reference, [22](#)
- rend, [31](#)
- reserve, [31](#)
- resize, [31](#)
- reverse_iterator, [22](#)
- shrink_to_fit, [31](#)
- size, [32](#)
- size_, [33](#)
- size_type, [22](#)
- swap, [32](#)
- value_type, [22](#)
- Vector, [23](#), [24](#)
- vector.h, [51](#)
- vector_testai.cpp, [53](#)
 - main, [54](#)
 - TEST_F, [54–57](#)
- VectorTest, [33](#)
 - expectEqual, [33](#)
 - SetUp, [33](#)
 - TearDown, [34](#)
- veiksmoPavadinimas
 - Laikas, [8](#)
- Zmogus, [34](#)
 - ~Zmogus, [36](#)
 - operator=, [36](#)
 - pavarde, [36](#)
 - pavarde_, [38](#)
 - print, [36](#)
 - read, [37](#)
 - setPavarde, [37](#)
 - setVardas, [37](#)
 - vardas, [37](#)
 - vardas_, [38](#)
 - Zmogus, [35](#)
- zmogus.cpp, [57](#)
 - operator<<, [58](#)
 - operator>>, [58](#)
- zmogus.h, [58](#)
 - operator<<, [59](#)
 - operator>>, [59](#)